

PARQUE CIENTÍFICO Y TECNOLÓGICO UTPL

ISBN Solutions – Plataforma Digital

Versión 1.0.0

Manual de usuario

Autores:
Líder de grupo: Byron Alvarez
Estudiante: Cody Cabrera
Estudiante: Byron Alvarez
Docente Tutor: Ing. René Elizalde

1. Capítulo 1: Generalidades del Proyecto

1.1. Introducción

El presente documento detalla el análisis y planificación para el desarrollo de una plataforma web de gestión de pedidos orientada a la comercialización de productos e insumos para micromercados. El sistema actúa como un puente tecnológico entre empresas mayoristas y tiendas locales (principalmente en zonas rurales), brindando oportunidades a vendedores independientes mediante un catálogo con inventario en tiempo real, incentivos por gamificación y un stack tecnológico moderno basado en Angular para el frontend y Django con PostgreSQL para el backend.

1.2. Problemática

Las empresas mayoristas y proveedores de micromercados en zonas rurales enfrentan una pérdida constante de ventas y clientes debido a un control deficiente y desfasado del inventario entre el punto de abastecimiento y la toma de pedidos en campo. Simultáneamente, la adopción de soluciones de compra directa (e-commerce) es inviable debido a la barrera tecnológica y desconfianza de los dueños de tiendas (población mayor a 60 años). Esto hace indispensable la figura de vendedores intermediarios, quienes, al no contar con información de stock en tiempo real ni incentivos dinámicos, generan incumplimientos en las entregas y estancamiento de productos.

1.3. Metodología o estrategia de desarrollo

Se empleará un enfoque ágil basado en Scrum, adaptado para el desarrollo iterativo e incremental del software. Esta metodología permitirá entregar módulos funcionales en ciclos cortos. La comunicación del equipo se gestionará mediante juntas de planificación de sprint, organizaciones diarias de sincronización (daily stand-ups) y juntas de revisión al finalizar cada iteración. Se utilizarán herramientas de control de versiones (Git) para el seguimiento de las tareas.

2. Capítulo 2: Planificación y Análisis

2.1. Análisis

El proyecto se enfoca en resolver la brecha de información entre el inventario de la comercializadora y el vendedor en campo. Los interesados principales incluyen a las Comercializadoras (quienes pagan la suscripción y proveen el inventario mediante integraciones API), los Vendedores (usuarios de la PWA que registran pedidos y ganan comisiones/puntos), y los dueños de Micromercados (beneficiarios indirectos). El alcance se limita a la intermediación y toma de pedidos, dejando fuera la logística de entrega física.

2.2. Requisitos funcionales

ID	Descripción
RF-01	El sistema debe permitir el registro y autenticación de usuarios mediante RUC (Comercializadoras y Vendedores).
RF-02	La plataforma debe exponer una API para que las comercializadoras actualicen su inventario en tiempo real.
RF-03	El sistema debe permitir al vendedor visualizar el catálogo de productos disponibles y registrar pedidos asignados a una tienda específica.
RF-04	El sistema debe calcular y mostrar las comisiones y puntos de gamificación obtenidos por el vendedor tras la confirmación de una venta.

2.3. Requisitos no funcionales

ID	Descripción
RNF-01	Arquitectura: El backend debe estar desarrollado en Django (Python) y el frontend en Angular.
RNF-02	Base de Datos: Se utilizará PostgreSQL para garantizar la integridad y concurrencia transaccional.
RNF-03	Usabilidad: La interfaz del vendedor debe ser responsiva (Mobile-First) para asegurar su correcto uso en trabajo de campo.

2.4. Historias de usuario / Casos de Uso

- HU-01 (Vendedor): Como vendedor, quiero ver el inventario actualizado en tiempo real para evitar vender productos que están fuera de stock y perder credibilidad con las tiendas.
- HU-02 (Comercializadora): Como administrador de la comercializadora, quiero conectar mi sistema de inventario a la plataforma mediante una API para que el stock se refleje automáticamente sin intervención manual.
- HU-03 (Vendedor): Como vendedor, quiero visualizar mis puntos acumulados en un dashboard para canjearlos y monitorear mis ganancias por comisiones.

2.5. Sprint

El desarrollo se dividirá en Sprints de 2 semanas:

1. Sprint 1: Diseño de base de datos en PostgreSQL, configuración de Django y creación del módulo de Autenticación/Roles.
2. Sprint 2: Desarrollo de los endpoints (API) de inventario y maquetación de la interfaz principal en Angular.
3. Sprint 3: Módulo de toma de pedidos, lógica transaccional de carritos de compras y sistema de cálculo de comisiones.
4. Sprint 4: Implementación del motor de gamificación (puntos por venta) y ajustes de usabilidad para dispositivos móviles.

2.6. Diagramas: Clases y BD (diseño lógico) - Preguntas y respuesta al diagrama

El diseño lógico en PostgreSQL contempla las siguientes entidades principales: Usuario, Vendedor, Comercializadora, Tienda, Producto, Inventario, Pedido, DetallePedido, y TransaccionPuntos.

A continuación, se plantean preguntas críticas para validar la robustez del modelo de clases de cara a su implementación en Django:

Pregunta 1 (Concurrencia): ¿Cómo resuelve el modelo de clases la concurrencia si dos vendedores intentan registrar un pedido para el mismo producto simultáneamente, y el stock disponible es insuficiente para ambos?

Respuesta: El modelo en Django debe implementar bloqueo optimista (Optimistic Locking) agregando un campo de version en la entidad Inventario. Antes de guardar el DetallePedido y descontar el stock, PostgreSQL verifica que la versión no haya cambiado; si cambió, se lanza una excepción y se notifica al vendedor que el stock acaba de agotarse.

Pregunta 2 (Acoplamiento de Pagos): Si el modelo de negocio cobra una comisión del 1 % al 2 % por venta, ¿dónde se registra esta obligación para no afectar el monto bruto que la tienda debe pagar a la comercializadora?

Respuesta: La clase Pedido registrará el monto_total.tienda. Se debe crear una clase auxiliar LiquidacionComercializadora que procese los pedidos en estado .Entregado y calcule el "fee" de la plataforma separando los montos de la cuenta por cobrar, manteniendo una trazabilidad contable limpia.

Pregunta 3 (Gamificación Dinámica): ¿Cómo se estructura la relación para que el sistema asigne recompensas variables (apuestas/puntos) según la urgencia de vender un producto específico?

Respuesta: Se debe evitar un atributo estático en Producto. En su lugar, se implementa una clase CampanaRecompensa que tiene una relación uno a muchos con Producto y posee fechas de vigencia. El modelo calcula los puntos cruzando el DetallePedido con la campaña activa en la fecha del pedido.

2.7. Diagrama de componentes

El sistema se compone de:

- Componente de Interfaz de Usuario (Angular): Módulos de Autenticación, Catálogo, Carrito y Dashboard de Vendedor.
- Componente API Gateway (Django REST Framework): Expone los endpoints seguros mediante JWT.
- Componente Core de Negocios (Django): Maneja la lógica de pedidos, cálculo de inventario y asignación de puntos.
- Componente de Persistencia (PostgreSQL): Almacena relaciones transaccionales y catálogos.
- Interfaces Externas: APIs RESTful preparadas para recibir webhooks o cargas JSON desde los ERP de las comercializadoras.

2.8. Arquitectura lógica y física (propuesta)

Arquitectura Lógica: Modelo Cliente-Servidor (SPA con Angular conectada a una API REST en Django).

Arquitectura Física:

- Capa Cliente: Navegadores web en dispositivos móviles y computadoras de escritorio.
- Capa de Presentación / Proxy: Servidor Nginx que sirve los archivos estáticos de Angular y actúa como proxy inverso.
- Capa de Aplicación: Contenedores Docker ejecutando Unicorn para servir la aplicación Django.
- Capa de Datos: Servidor de base de datos PostgreSQL gestionado, con copias de seguridad automáticas.

2.9. Prototipo de pantallas no funcional

- Pantalla de Login: Ingreso seguro con RUC y contraseña.
- Dashboard del Vendedor: Resumen rápido del presupuesto de ventas, puntos de gamificación acumulados y comisiones pendientes.
- Catálogo de Inventario: Lista de productos con imágenes, precio mayorista y un indicador visual claro (verde/rojo) del stock disponible en tiempo real.

- Registro de Pedido (Checkout): Selección del cliente (Tienda/Micromercado), resumen del carrito y botón de confirmación de pedido.